

# Production RAG Indexing Pipelines

Fingerprinting · Idempotent Upsert · Batch Scale · Multi-Tenant · Live Schema · Fine-Tuning

WEEK 3 · DAY 16

Tuesday, July 21 2026

Day 16 of 84

| Sec    | Topic                   | Coverage                                                         |
|--------|-------------------------|------------------------------------------------------------------|
| Sec 1  | Pipeline Architecture   | Source → change detect → chunk → embed → upsert                  |
| Sec 2  | Document Fingerprinting | SHA-256 hash, DocumentRegistry, needs_reindex()                  |
| Sec 3  | Idempotent Upsert       | Delete old chunks before writing new — no ghost chunks           |
| Sec 4  | Batch Indexing at Scale | Thread pool chunking + GPU batching + async upsert               |
| Sec 5  | Metadata + Multi-Tenant | tenant_id filter, Option A vs B isolation strategies             |
| Sec 6  | Live Schema Changes     | SchemaChangeDetector, LiveSchemaIndexer, CDC vs poll             |
| Sec 7  | Embedding Fine-Tuning   | MNRL loss, SQL (question, DDL) pairs, +9pp recall@3              |
| Sec 8  | Index Maintenance       | HNSW rebuild, Qdrant status, nightly reindex job                 |
| Sec 9  | Production Indexing API | FastAPI: /index/schema, /delete, /health, background tasks       |
| Sec 10 | FAANG Patterns          | Google/Microsoft/Amazon/Meta/Netflix indexing strategies         |
| Sec 11 | Architect's Lens        | Production checklist: correctness, perf, reliability, monitoring |
| Sec 12 | Exercises + Summary     | Fingerprint test, schema migration sim, lag measurement          |

## Section 1 — Indexing Pipeline Architecture

Production indexing handles the full document lifecycle: new documents arrive, existing ones update, outdated ones are deleted, and schemas change — all without downtime or index staleness. Three failure modes to prevent:

### FAILURE 1 — Stale index

Documents updated in source but vector index not refreshed.  
RAG retrieves old version → model answers from outdated data.

### FAILURE 2 — Ghost chunks

Document deleted from source but chunks remain in vector store.  
RAG retrieves deleted content → confidently generates wrong answer.

### FAILURE 3 — Index drift under load

High write throughput causes embedding backlog.  
New documents not queryable for minutes → missing-context errors.

### PIPELINE FLOW:

```
Source systems (PostgreSQL, Confluence, GitHub)
  ↓ change events (poll / CDC / webhook)
Change Detector → fingerprint (SHA-256 hash)
  ↓ only changed docs
Document Preprocessor → clean, dedup
  ↓
Chunker → strategy per doc type
  ↓
Embedder → batch GPU, async
  ↓
Idempotent Upsert → delete old chunks, write new chunks
  ↓
Vector Store (Qdrant) + Document Registry (PostgreSQL)
```

## Section 2 — Document Fingerprinting and Change Detection

SHA-256 hashing of document content enables skip-if-unchanged logic. The DocumentRegistry stores the hash, chunk IDs, and metadata for every indexed document in PostgreSQL. On each pipeline run, only documents whose content hash has changed are re-embedded — saving 99%+ of compute.

```
import hashlib, json, time

class DocumentRegistry:
    """Tracks indexed documents and their content hashes in PostgreSQL."""
    def __init__(self, db_conn):
        self.conn = db_conn
        self.conn.execute("""
            CREATE TABLE IF NOT EXISTS doc_registry (
                doc_id          TEXT PRIMARY KEY,
                source          TEXT,
                content_hash    TEXT,          -- SHA-256 of raw content
                chunk_ids       JSONB,        -- Qdrant point IDs for all chunks
                metadata        JSONB,
                indexed_at      DOUBLE PRECISION,
```

```

        version          INT DEFAULT 1
    )
    """
)

def content_hash(self, content: str) -> str:
    return hashlib.sha256(content.encode()).hexdigest()

def needs_reindex(self, doc_id: str, current_content: str) -> bool:
    """True if document is new OR content has changed."""
    row = self.conn.execute(
        "SELECT content_hash FROM doc_registry WHERE doc_id = %s", (doc_id,)
    ).fetchone()
    if row is None: return True          # new document
    return row[0] != self.content_hash(current_content)

def register(self, doc_id, source, content, chunk_ids, metadata):
    self.conn.execute("""
        INSERT INTO doc_registry
            (doc_id, source, content_hash, chunk_ids, metadata, indexed_at, version)
        VALUES (%s, %s, %s, %s::jsonb, %s::jsonb, %s, 1)
        ON CONFLICT (doc_id) DO UPDATE
            SET content_hash = EXCLUDED.content_hash,
                chunk_ids    = EXCLUDED.chunk_ids,
                metadata     = EXCLUDED.metadata,
                indexed_at   = EXCLUDED.indexed_at,
                version      = doc_registry.version + 1
    """, (doc_id, source, self.content_hash(content),
        json.dumps(chunk_ids), json.dumps(metadata), time.time()))

def get_chunk_ids(self, doc_id: str) -> list[str] | None:
    row = self.conn.execute(
        "SELECT chunk_ids FROM doc_registry WHERE doc_id = %s", (doc_id,)
    ).fetchone()
    return json.loads(row[0]) if row else None

```

## Section 3 — Idempotent Upsert: No Ghost Chunks

Before writing new chunks, delete the document's old chunks. This prevents ghost chunks — stale embeddings from a previous version that would confuse retrieval with outdated content.

```
class ProductionIndexer:
    def index_document(self, doc_id, source, content, chunks, metadata) -> dict:
        """
        Idempotent: skip if unchanged, replace if changed, clean on delete.
        Returns: {"action": "skipped"|"created"|"updated", "n_chunks": int}
        """
        if not self.registry.needs_reindex(doc_id, content):
            return {"action": "skipped", "n_chunks": 0}

        # Step 1: delete old chunks → prevents ghost chunks accumulating
        old_ids = self.registry.get_chunk_ids(doc_id)
        if old_ids:
            self.client.delete(collection_name=self.collection,
                              points_selector=old_ids)
            action = "updated"
        else:
            action = "created"

        # Step 2: embed all chunks in one GPU batch call
        vectors = self.embedder.encode(chunks, batch_size=64,
                                       normalize_embeddings=True)

        # Step 3: upsert new chunks with full metadata payload
        new_ids = [str(uuid.uuid4()) for _ in chunks]
        points = [
            PointStruct(
                id=pid, vector=vec.tolist(),
                payload={**metadata, "doc_id": doc_id, "source": source,
                          "chunk_idx": i, "text": chunk})
                for i, (pid, vec, chunk) in enumerate(zip(new_ids, vectors, chunks))
            ]
        self.client.upsert(collection_name=self.collection, points=points)

        # Step 4: register in PostgreSQL (atomic – hash + chunk IDs together)
        self.registry.register(doc_id, source, content, new_ids, metadata)
        return {"action": action, "n_chunks": len(chunks)}

    def delete_document(self, doc_id: str):
        """Hard delete: remove from vector store AND registry."""
        old_ids = self.registry.get_chunk_ids(doc_id)
        if old_ids:
            self.client.delete(collection_name=self.collection,
                              points_selector=old_ids)
        self.registry.conn.execute(
            "DELETE FROM doc_registry WHERE doc_id = %s", (doc_id,))
```

## Section 4 — Batch Indexing at Scale

For initial indexing of large corpora (millions of documents): parallel chunking in a thread pool (CPU-bound) + single batched GPU embedding call + async upsert (I/O-bound). The bottleneck is

always the GPU embedding step.

```
import asyncio, concurrent.futures
from itertools import islice

def batched(it, n):
    it = iter(it)
    while batch := list(islice(it, n)): yield batch

class BatchIndexingPipeline:
    async def run(self, documents: list[dict]) -> dict:
        # Filter unchanged documents first (saves 99%+ compute)
        to_index = [d for d in documents
                     if self.indexer.registry.needs_reindex(d["doc_id"], d["content"])]
        skipped = len(documents) - len(to_index)

        # Chunk in thread pool (CPU-bound, parallelise across cores)
        loop = asyncio.get_event_loop()
        chunker = RecursiveCharacterTextSplitter(chunk_size=1000, chunk_overlap=200)
        with concurrent.futures.ThreadPoolExecutor(self.chunk_workers) as pool:
            all_chunks = await asyncio.gather(*[
                loop.run_in_executor(pool, chunker.split_text, d["content"])
                for d in to_index
            ])

        # Single batched GPU embedding call across ALL chunks
        flat_texts = [c for chunks in all_chunks for c in chunks]
        flat_vectors = self.indexer.embedder.encode(
            flat_texts, batch_size=self.embed_batch,
            normalize_embeddings=True, show_progress_bar=True)

        # Upsert in batches of 256 (optimal for Qdrant)
        idx = 0
        for doc, doc_chunks in zip(to_index, all_chunks):
            n = len(doc_chunks)
            vecs = flat_vectors[idx:idx+n]; idx += n
            old = self.indexer.registry.get_chunk_ids(doc["doc_id"])
            if old: self.indexer.client.delete(self.indexer.collection, old)
            new_ids = [str(uuid.uuid4()) for _ in doc_chunks]
            pts = [PointStruct(id=pid, vector=v.tolist(),
                              payload={**doc["metadata"], "doc_id":doc["doc_id"],
                                      "chunk_idx":i, "text":c})
                  for i,(pid,v,c) in enumerate(zip(new_ids,vecs,doc_chunks))]
            for batch in batched(pts, 256):
                self.indexer.client.upsert(self.indexer.collection, points=batch)
            self.indexer.registry.register(doc["doc_id"],doc["source"],
                                          doc["content"],new_ids,doc["metadata"])
        return {"skipped":skipped,"indexed":len(to_index)}

# Throughput on A100 (bge-large-en-v1.5, batch=128):
# Chunking (8 threads): ~50,000 docs/min
# Embedding (GPU): ~8,000 chunks/min ← bottleneck
# Upsert (batch=256): ~20,000 chunks/min
```

## Section 5 — Metadata Filtering and Multi-Tenant Isolation

Every chunk payload carries `tenant_id` and `schema_name`. Every search applies a filter — users NEVER see across tenant boundaries.

```
# PAYLOAD SCHEMA — every chunk carries:
{
  "tenant_id": "acme_corp",      # strict isolation field
  "schema_name": "ecommerce",    # within-tenant namespace
  "doc_id": "schema_v42",
  "source": "postgresql",
  "table_name": "orders",
  "chunk_idx": 3,
  "version": 42,
  "indexed_at": 1752878400.0,
  "text": "CREATE TABLE orders ...",
}

# FILTER PATTERNS:
def tenant_filter(tenant_id: str) -> Filter:
    return Filter(must=[FieldCondition(key="tenant_id",
                                       match=MatchValue(value=tenant_id))])

def schema_filter(tenant_id: str, schema_name: str) -> Filter:
    return Filter(must=[
        FieldCondition(key="tenant_id", match=MatchValue(value=tenant_id)),
        FieldCondition(key="schema_name", match=MatchValue(value=schema_name)),
    ])

def recency_filter(tenant_id: str, days: int=30) -> Filter:
    cutoff = (datetime.now() - timedelta(days=days)).timestamp()
    return Filter(must=[
        FieldCondition(key="tenant_id", match=MatchValue(value=tenant_id)),
        FieldCondition(key="indexed_at", range=Range(gte=cutoff)),
    ])
```

| Option | Architecture                       | Best For           | Isolation                       | Ops Cost |
|--------|------------------------------------|--------------------|---------------------------------|----------|
| A      | Single collection + payload filter | < 50 tenants       | Good (filter-based)             | Low      |
| B      | One collection per tenant          | > 50 tenants       | Strong (no cross-contamination) | High     |
| C      | Qdrant namespaces (v1.9+)          | Strict SLA tenants | Strongest (sub-graph HNSW)      | Medium   |

## Section 6 — Live Schema Change Handling

The hardest production problem for SQL schema RAG: tables are added, columns modified, tables dropped. The vector index must reflect this in near-real-time without re-indexing the entire schema.

```

class SchemaChangeDetector:
    """Poll PostgreSQL information_schema for structural changes."""
    def __init__(self, pg_conn, poll_interval_sec: int=60):
        self.conn = pg_conn; self.poll_interval = poll_interval_sec
        self._cache: dict[str, str] = {} # table_key → column signature

    def get_current_ddl(self, schema_name: str) -> dict[str, str]:
        rows = self.conn.execute("""
            SELECT table_name,
                   string_agg(column_name || ' ' || data_type,
                               ', ' ORDER BY ordinal_position) AS columns
            FROM information_schema.columns WHERE table_schema = %s
            GROUP BY table_name
        """, (schema_name,)).fetchall()
        return {r[0]: r[1] for r in rows}

    def detect_changes(self, schema_name: str) -> list[SchemaChange]:
        current = self.get_current_ddl(schema_name)
        changes = []
        for table, ddl in current.items():
            key = f"{schema_name}.{table}"
            if key not in self._cache:
                changes.append(SchemaChange("table_added", table, schema_name,
                                             f"CREATE TABLE {table} ({ddl});"))
            elif self._cache[key] != ddl:
                changes.append(SchemaChange("table_modified", table, schema_name,
                                             f"CREATE TABLE {table} ({ddl});"))
        for key in self._cache:
            sn, tn = key.split(".",1)
            if sn==schema_name and tn not in current:
                changes.append(SchemaChange("table_dropped", tn, schema_name, None))
        # Update cache
        self._cache = {f"{schema_name}.{t}": d for t,d in current.items()}
        return changes

class LiveSchemaIndexer:
    def apply_changes(self, schema_name: str, tenant_id: str):
        for change in self.detector.detect_changes(schema_name):
            doc_id = f"{tenant_id}/{schema_name}/{change.table_name}"
            if change.change_type == "table_dropped":
                self.indexer.delete_document(doc_id)
            else:
                self.indexer.index_document(
                    doc_id=doc_id, source="postgresql",
                    content=change.new_sql,
                    chunks=[f"Table: {change.table_name}\n{change.new_sql}"],
                    metadata={"tenant_id":tenant_id,"schema_name":schema_name,
                             "table_name":change.table_name,"source":"postgresql"})

# Alternative: CDC via PostgreSQL logical replication (pg_logical)
# - Event-driven (sub-second latency) vs polling (60s latency)
# - Requires wal_level=logical in postgresql.conf
# - Use Debezium or pg_logical plugin → Kafka → consumer → indexer

```

## Section 7 — Embedding Model Fine-Tuning for SQL Domain

Off-the-shelf BGE/E5 models are trained on general text. Fine-tuning on (question, table DDL) pairs improves SQL schema retrieval by ~9 percentage points at recall@3, with only 30 minutes of A100 training.

```
from sentence_transformers import SentenceTransformer, InputExample, losses
from sentence_transformers.evaluation import InformationRetrievalEvaluator
from torch.utils.data import DataLoader

TRAIN_PAIRS = [
    ("how many users signed up last month",
     "CREATE TABLE users (user_id INT, email TEXT, created_at DATE)"),
    ("total revenue from completed orders",
     "CREATE TABLE orders (order_id INT, amount FLOAT, status TEXT)"),
    ("which products are in electronics category",
     "CREATE TABLE products (product_id INT, name TEXT, category TEXT, price FLOAT)"),
    ("find users who never placed an order",
     "CREATE TABLE users (user_id INT); CREATE TABLE orders (order_id INT, user_id INT)"),
    # ... 500-1000 total pairs for good fine-tuning
]

def fine_tune_schema_embedder(base_model="BAAI/bge-large-en-v1.5",
                              output_dir="./sql-schema-embedder", epochs=3):
    model = SentenceTransformer(base_model)

    # MultipleNegativesRankingLoss (MNRL) = InfoNCE with in-batch negatives
    # batch_size=32 → 1 positive + 31 negatives per example (free from batch)
    loader = DataLoader([InputExample(texts=[q,d]) for q,d in TRAIN_PAIRS],
                        batch_size=32, shuffle=True)
    loss = losses.MultipleNegativesRankingLoss(model=model)

    # Evaluator: recall@1, recall@5, MRR on held-out queries
    evaluator = InformationRetrievalEvaluator(
        queries={"q1":"count all users","q2":"sum of order amounts"},
        corpus={"users":"CREATE TABLE users (user_id INT, email TEXT)",
                "orders":"CREATE TABLE orders (order_id INT, amount FLOAT)"},
        relevant_docs={"q1":{"users"},"q2":{"orders"}},
        main_score_function="cos_sim",
    )
    model.fit(train_objectives=[(loader,loss)], epochs=epochs,
              warmup_steps=100, evaluator=evaluator, evaluation_steps=200,
              output_path=output_dir, save_best_model=True)
    return model

# Expected improvement (1000 training pairs, 3 epochs, A100):
# Recall@3: base bge-large = ~82% → fine-tuned = ~91% (+9pp)
# Training: ~30 minutes on 1x A100
# Inference: same speed as base model (fine-tuning doesn't change architecture)
```

| Stage                    | Recall@3 | Cost | Notes                              |
|--------------------------|----------|------|------------------------------------|
| Off-the-shelf all-MiniLM | ~72%     | Free | Fast CPU inference; lowest quality |
| Off-the-shelf bge-large  | ~82%     | Free | Best OSS starting point            |



|                            |      |                  |                                   |
|----------------------------|------|------------------|-----------------------------------|
| Fine-tuned bge-large (SQL) | ~91% | \$2-5 A100-hour  | +9pp; 500-1000 training pairs     |
| text-embedding-3-large     | ~88% | \$0.13/1M tokens | Managed; no fine-tuning available |

## Section 8 — Index Maintenance

```

from qdrant_client.models import HnswConfigDiff, OptimizersConfigDiff

# 1. Monitor collection health
info = client.get_collection("sql_schemas")
print(f"Points:    {info.points_count}")
print(f"Indexed:    {info.indexed_vectors_count}")
print(f"Status:      {info.status}") # green=OK, yellow=indexing, red=error

# 2. Tune HNSW parameters (after large bulk inserts)
client.update_collection("sql_schemas",
    hnsw_config=HnswConfigDiff(
        m=32,                # connections per node (default 16; higher = better recall)
        ef_construct=200,    # build-time beam width (higher = better recall, slower)
        full_scan_threshold=10000, # brute-force below this count
    ),
    optimizer_config=OptimizersConfigDiff(
        indexing_threshold=20000, # build HNSW index after this many vectors
        memmap_threshold=50000,  # move to mmap storage above this count
    ))

# 3. Force rebuild after large deletion batches (> 20% of index)
client.update_collection("sql_schemas",
    optimizer_config=OptimizersConfigDiff(indexing_threshold=0))
# indexing_threshold=0 triggers immediate HNSW rebuild of all segments

# 4. Nightly reindex job (stale documents older than 7 days)
import schedule

def nightly_reindex():
    old_docs = pg_conn.execute("""
        SELECT doc_id, source FROM doc_registry
        WHERE indexed_at < extract(epoch from now()) - 604800 -- 7 days
        AND source = 'postgresql'
    """).fetchall()
    print(f"Nightly reindex: {len(old_docs)} stale documents")
    for doc_id, _ in old_docs:
        refresh_document(doc_id) # re-fetch from source + reindex

schedule.every().day.at("02:00").do(nightly_reindex)

# 5. Index lag metric (Prometheus / Grafana alert)
lag_sec = pg_conn.execute("""
    SELECT extract(epoch from now()) - min(indexed_at) FROM doc_registry
""").fetchone()[0]
print(f"Max index lag: {lag_sec:.0f}s")
# Alert if lag > 300 seconds (5-minute SLO)

```

## Section 9 — Production Indexing API (FastAPI)

```

from fastapi import FastAPI, BackgroundTasks
from pydantic import BaseModel

app = FastAPI(title="RAG Indexing API")

@app.on_event("startup")
async def startup():
    app.state.indexer = ProductionIndexer("http://localhost:6333",
                                         "sql_schemas", get_pg_conn())
    app.state.detector = SchemaChangeDetector(get_pg_conn())
    app.state.live = LiveSchemaIndexer(app.state.indexer, app.state.detector)
    asyncio.create_task(watch_schemas_loop())  # background: poll every 60s

async def watch_schemas_loop():
    while True:
        for tenant_id, schema_name in WATCHED_SCHEMAS:
            app.state.live.apply_changes(schema_name, tenant_id)
            await asyncio.sleep(60)

class IndexRequest(BaseModel):
    tenant_id: str
    schema_name: str
    schema_sql: str

@app.post("/index/schema")
async def index_schema(req: IndexRequest, bg: BackgroundTasks):
    """Trigger immediate reindex (e.g., called on DB migration completion)."""
    def _do():
        for chunk in chunk_schema(req.schema_sql):
            doc_id = f"{req.tenant_id}/{req.schema_name}/{chunk['table']}"
            app.state.indexer.index_document(
                doc_id=doc_id, source="postgresql", content=chunk["sql"],
                chunks=[f"Table: {chunk['table']}\n{chunk['sql']}"],
                metadata={"tenant_id": req.tenant_id, "schema_name": req.schema_name,
                        "table_name": chunk["table"], "source": "postgresql"})
    bg.add_task(_do)
    return {"status": "indexing", "schema": req.schema_name}

@app.delete("/index/schema/{tenant_id}/{schema_name}")
async def delete_schema(tenant_id: str, schema_name: str):
    """Remove all chunks for a tenant/schema (offboarding)."""
    app.state.indexer.client.delete("sql_schemas",
                                    points_selector=schema_filter(tenant_id, schema_name))
    app.state.indexer.registry.conn.execute(
        "DELETE FROM doc_registry WHERE metadata->>'tenant_id'=%s "
        "AND metadata->>'schema_name'=%s", (tenant_id, schema_name))
    return {"status": "deleted"}

@app.get("/index/health")
async def index_health():
    info = app.state.indexer.client.get_collection("sql_schemas")
    return {"points": info.points_count, "status": str(info.status)}

```

## Section 10 — FAANG Production Indexing Patterns

### Google — Two-Stage Write with Quality Gate (Vertex AI Search)

Pub/Sub as change event bus: Drive/Docs/Sheets publishes change events. Cloud Run worker subscribes, fetches, chunks, embeds (Gecko model), upserts. Two-stage write: new chunks → staging index first (queryable, lower priority). After quality check (embedding norm, PII scan, content safety) → primary index. Prevents corrupted or unsafe documents from appearing in live search instantly.

### Microsoft — Delta Processing with Azure Data Factory (M365 Copilot)

ADF pipelines with watermark timestamps: only docs modified after last watermark are processed each run (delta processing). Sources: SharePoint, Teams, Exchange. Pipeline: extract → clean (remove HTML boilerplate) → chunk (structure-aware) → embed (Ada-002) → upsert to Azure Cognitive Search. Runs every 15 minutes. ADF pipeline is idempotent: restart from last watermark on failure.

### Amazon — Event-Driven with SQS + Lambda (Amazon Q / Kendra)

SQS as indexing queue: Salesforce, Confluence connectors publish document IDs. Lambda (triggered by SQS) fetches, preprocesses, calls Kendra index API. Lambda concurrency scales with queue depth automatically — handles burst of 10,000 simultaneous updates via SQS buffering. Dead-letter queue captures documents failing after 3 retries.

### Meta — Batch Indexing with Manifold + Async Checkpointing

Weekly batch for primary knowledge base; hourly incremental for high-churn sources. Batch job on Manifold data platform: checkpoints every 1M documents. Job resumes from last checkpoint on failure (not from scratch). Embedding model: fine-tuned DPR on internal search query logs — same (question, clicked document) signal used for web search ranking.

### Netflix — Real-Time Content Metadata Indexing (< 15 min SLA)

New title ingested → metadata extraction LLM runs on synopsis (async, ~5 min) → structured metadata to Cassandra → Kafka change event → consumer re-embeds full title record → upsert to vector store. SLA: new titles retrievable by recommendation RAG within 15 minutes. Index lag monitored via Kafka consumer lag + Prometheus alert if lag > 10 min.



## Section 11 — Architect's Lens: Production Checklist

### PRODUCTION INDEXING CHECKLIST:

#### CORRECTNESS

- [ ] Document fingerprinting: SHA-256 hash prevents unnecessary reindex
- [ ] Idempotent upsert: delete old chunks BEFORE writing new ones
- [ ] Hard deletes: remove from vector store AND registry atomically
- [ ] Metadata payload: tenant\_id, schema\_name, source, indexed\_at on every chunk

#### PERFORMANCE

- [ ] Bulk load: parallel chunking (thread pool) + batched GPU embedding
- [ ] Embedding bottleneck: MiniLM (384d) is 10x faster than bge-large (1024d)
- [ ] Upsert batch size: 256 points per call (optimal for Qdrant)
- [ ] Change detection: hash check BEFORE any embedding call (saves 99%+ compute)

#### RELIABILITY

- [ ] Dead-letter queue or retry table for failed indexing jobs
- [ ] Checkpoint batch jobs > 100K documents
- [ ] CDC (pg\_logical) for < 5s schema change latency; polling for > 60s tolerance
- [ ] HNSW rebuild scheduled after > 20% deletion (fragmentation)

#### MULTI-TENANCY

- [ ] tenant\_id in every payload; applied as filter on every search
- [ ] Option A (single collection + filter) for < 50 tenants
- [ ] Option B (per-tenant collection) for > 50 or strict isolation
- [ ] Offboarding: delete by payload filter + purge doc\_registry

#### EMBEDDING MODEL UPGRADE PATH

- [ ] Start: bge-large-en-v1.5 (free, ~82% recall@3 on SQL schemas)
- [ ] After 500+ (question, DDL) pairs: fine-tune → ~91% recall@3
- [ ] Stamp embedding model version in metadata → forced reindex on model update
- [ ] Run InformationRetrievalEvaluator weekly to detect recall drift

#### MONITORING

- [ ] Qdrant collection status: green (indexed) vs yellow (indexing in progress)
- [ ] Index lag SLO: max(now - indexed\_at) < 5 minutes
- [ ] Dead-letter queue alert: depth > 0
- [ ] Recall@3 regression test: run weekly on curated eval set (> 85% pass)

## Section 12 — Exercises and Summary

### Exercise 1: Fingerprint and Skip Unchanged

```
registry = DocumentRegistry(conn)
docs = [{"doc_id":f"t{i}", "content":f"CREATE TABLE t{i}(id INT)"} for i in range(5)]
for d in docs: registry.register(d["doc_id"], "pg", d["content"], [], {})

# Modify 2 of 5 documents
docs[0]["content"] = "CREATE TABLE t0 (id INT, name TEXT)"
docs[2]["content"] = "CREATE TABLE t2 (id INT, value FLOAT)"

results = [registry.needs_reindex(d["doc_id"], d["content"]) for d in docs]
print(results) # [True, False, True, False, False] – only 2 reindexed
```

### Exercise 2: Simulate Schema Migration

```
old = "CREATE TABLE orders (order_id INT, user_id INT, amount FLOAT);"
new = "CREATE TABLE orders (order_id INT, user_id INT, amount FLOAT, status TEXT);"

indexer.index_document("acme/ecommerce/orders", "pg", old,
    [f"Table: orders\n{old}"], {"tenant_id": "acme", "table_name": "orders"})

result = indexer.index_document("acme/ecommerce/orders", "pg", new,
    [f"Table: orders\n{new}"], {"tenant_id": "acme", "table_name": "orders"})
print(result) # {"action": "updated", "n_chunks": 1}
# Verify: only 1 chunk for 'orders' in Qdrant (no duplicates)
```

### Summary

| Concept                 | Key Point                                               |
|-------------------------|---------------------------------------------------------|
| 3 failure modes         | Stale index, ghost chunks, index drift under load       |
| Document fingerprinting | SHA-256 → skip unchanged (saves 99%+ embedding compute) |
| Idempotent upsert       | Delete old chunks BEFORE writing new — no ghost chunks  |
| Batch pipeline          | Thread pool chunking + batched GPU embed + async upsert |
| Metadata filtering      | tenant_id + schema_name on every chunk and every search |
| Multi-tenant options    | Option A (< 50 tenants); Option B (> 50 or strict)      |
| Live schema changes     | Poll info_schema (60s) or CDC/pg_logical (< 5s)         |
| Embedding fine-tuning   | MNRL loss, SQL (question, DDL) pairs → +9pp recall@3    |
| HNSW maintenance        | Rebuild after > 20% deletion; m=32, ef_construct=200    |
| Index lag SLO           | max(now - indexed_at) < 5 minutes for live sources      |

**NEXT: Day 17 — RAG Retrieval Deep Dive**

Advanced retrieval: parent-child chunking, multi-vector indexing, query expansion, contextual compression, and building the hybrid retrieval layer for the Text-to-SQL system (Capstone Piece 2 completion).

## Eagle Eye Addendum — Day 16: Schema Chunking

Schema chunking is the strategy for splitting database schema (DDL) into units for indexing in the vector store. The key decision: chunk at the table level (one chunk per CREATE TABLE) vs. column level (one chunk per column). Table-level is standard.

# Schema chunking strategies for RAG indexing (Day 15-16 pipeline)

# STRATEGY 1: Table-level schema chunking (standard, used in this pipeline)

# Each chunk = one complete CREATE TABLE statement.

# Pros: preserves column relationships; model sees full table context.

# Cons: large tables (50+ columns) may exceed context window.

```
def chunk_schema_by_table(ddl: str) -> list[dict]:
    """Split DDL into one chunk per CREATE TABLE. Schema chunking at table level."""
    import re
    chunks = []
    for match in re.finditer(
        r"(CREATE TABLE \w+\s*\([^;]+\);)", ddl, re.IGNORECASE | re.DOTALL):
        table_ddl = match.group(1).strip()
        table_name = re.search(r"CREATE TABLE (\w+)", table_ddl, re.I).group(1)
        chunks.append({"table": table_name, "sql": table_ddl,
            "chunk_type": "table_schema"})
    return chunks
```

# STRATEGY 2: Column-level schema chunking (for very wide tables)

# Each chunk = one column definition + parent table context.

# Pros: fine-grained retrieval; can surface specific columns.

# Cons: loses intra-table column relationships; 10x more chunks.

# USED IN PRODUCTION (this pipeline):

# Schema chunking = table-level (one CREATE TABLE per vector store doc)

# Metadata: {table\_name, schema, column\_count, primary\_key, foreign\_keys}

# Embedding = table DDL text → BAAI/bge-large-en-v1.5 → 1024-dim vector